Pass-by-Value: Java's Most Misunderstood Rule

11 min read Updated 28 Jul 2026 Topic 5 of 65

Bookmark Mark done

Why it matters

This is the topic where confident candidates fail. Ask "is Java pass-by-value or pass-by-reference?" and a large fraction will answer "pass-by-value for primitives, pass-by-reference for objects" — which sounds sophisticated, is widely repeated, and is **wrong**. Java is pass-by-value. Always. Without exception. The confusion arises because when you pass an object, the value being copied is a *reference*, and that produces behaviour which superficially resembles pass-by-reference until you test the one case that distinguishes them.

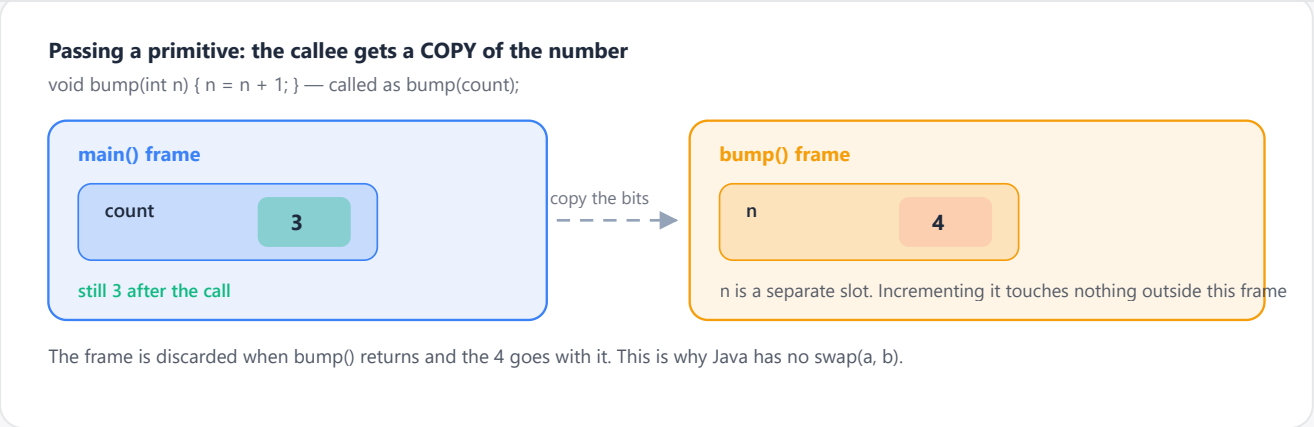
Interviewers love it because the wrong answer is so common that getting it right is a genuine signal — and because the correct explanation requires the previous two topics to be solid. It matters in practice too: it is why you cannot write `swap(a, b)` in Java, and why a method that "clears" the list you passed it sometimes works and sometimes does nothing.

The concept, simply

The rule, stated once, exactly:

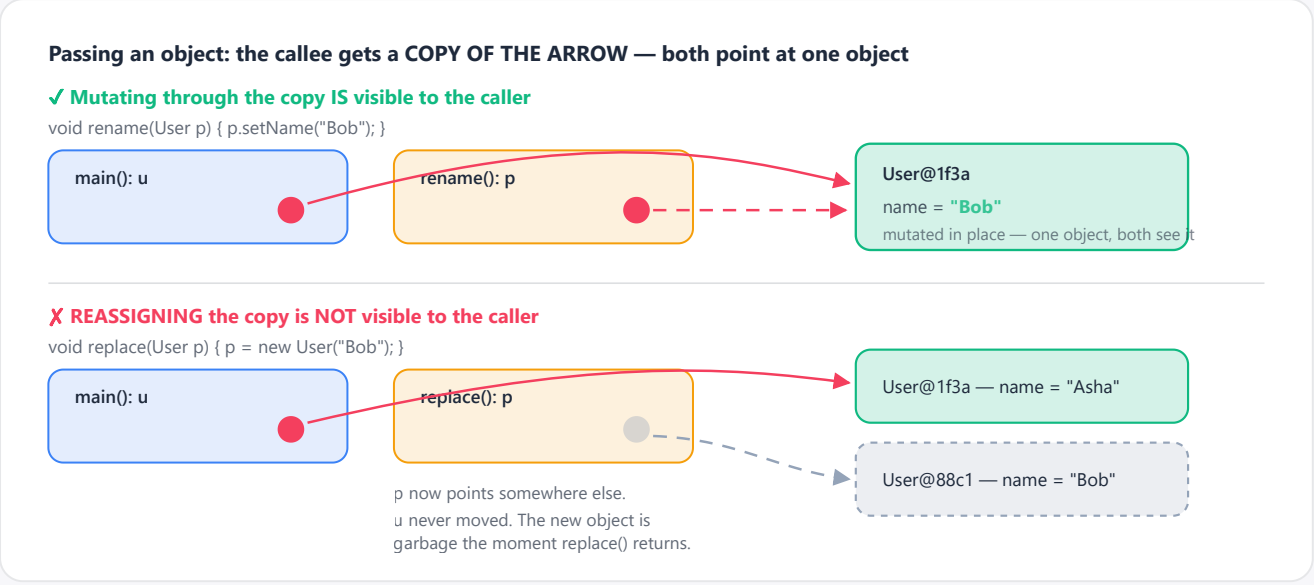
> **Java always passes arguments by value. When the argument is an object, the value copied is the reference — not the object.**

Unpack that. Passing an `int` copies the number into the callee's frame; two independent slots exist. Passing a `User` copies the *arrow* into the callee's frame; two



Primitives: the callee gets its own copy of the number. Whatever it does to that copy dies with the frame.

That shared object is where the illusion comes from. Because both arrows point at one object, *mutating* through either one is visible to both. But *reassigning* the callee's arrow only moves the callee's arrow. If Java were genuinely pass-by-reference, reassignment inside the method would change the caller's variable too — and it does not.



The decisive test. Mutation through the copied arrow is visible to the caller (top). Reassigning the copied arrow is not (bottom) — which is precisely what proves Java is not pass-by-reference.

How it works

Mechanically it is simple once you picture stack frames. When a method is invoked, the JVM pushes a new frame and copies each argument's bits into that frame's local-variable slots. Copying `int 3` copies the number. Copying a reference copies the

So there are exactly two behaviours to keep straight:

Inside the method you…	Caller sees the change?	Why
`p.setName("Bob")` — mutate the object	**Yes**	One object, reached through both arrc
`p = new User("Bob")` — reassign the parameter	**No**	Only the callee's own slot was rebounc
`n = n + 1` on an `int`	**No**	The callee's slot holds a copy of the nu
`arr[0] = 99` on an array	**Yes**	Arrays are objects; this mutates the sh
`s = s + "!"` on a `String`	**No**	`String` is immutable, so `+` creates

The `String` row is the one that convinces people Java must be pass-by-value, because it is the most familiar case where "I changed it in the method and nothing happened".

Code examples

The two object cases, side by side — this is the whole topic:

JAVA Copy

```java
class User {
    String name;
    User(String name) { this.name = name; }
}
```

```java
    // MUTATION — reaches through the copied arrow to the shared object.
    static void rename(User p) {
        p.name = "Bob";          // caller sees this
    }

    // REASSIGNMENT — rebinds only this frame's slot.
    static void replace(User p) {
        p = new User("Bob");     // caller sees NOTHING
        p.name = "Carol";        // and this touches only the new, doomed
    }

    public static void main(String[] args) {
        User u = new User("Asha");

        rename(u);
        System.out.println(u.name);   // Bob    ← mutation was visible

        User v = new User("Asha");
        replace(v);
        System.out.println(v.name);   // Asha   ← reassignment was NOT vis
    }
}
```

If Java were pass-by-reference, the second print would say `Carol`. It says `Asha`. That single line is the proof, and it is the fastest way to win the argument in an interview.

The famous consequence — you cannot write `swap`:

JAVA

Copy

```java
// Does nothing. Both parameters are copies; swapping them
// rearranges only this frame's two slots.
static void swap(int a, int b) {
    int t = a; a = b; b = t;
}

// Also does nothing to the caller's variables — same reason, one level up
static void swap(User a, User b) {
    User t = a; a = b; b = t;
}
```

```java
// container", because rebinding a caller's variable is simply not availab
static int[] swapped(int a, int b) { return new int[]{ b, a }; }
```

Collections behave exactly as the rule predicts, and this catches people in real code:

JAVA Copy

```java
static void addItem(List<String> list) {
    list.add("added");          // ✓ mutates the shared list — caller sees
}

static void replaceList(List<String> list) {
    list = new ArrayList<>();   // ✗ rebinds the local slot only
    list.add("invisible");      // caller's list is untouched
}

static void clearList(List<String> list) {
    list.clear();               // ✓ mutation again — caller's list IS empt
}

List<String> items = new ArrayList<>(List.of("one"));
addItem(items);      System.out.println(items);  // [one, added]
replaceList(items);  System.out.println(items);  // [one, added]  ← unchan
clearList(items);    System.out.println(items);  // []
```

And the `String` case, which surprises people precisely because it looks like mutation:

JAVA Copy

```java
static void shout(String s) {
    s = s + "!";                // creates a NEW String, rebinds the local
}

static void shout(StringBuilder sb) {
    sb.append("!");             // MUTATES the shared object
}

String s = "hi";
```

```
StringBuilder sb = new StringBuilder("hi");
shout(sb);
System.out.println(sb);        // hi!  ← mutable object, mutation visible
```

Under the hood

What would pass-by-reference actually look like? The cleanest way to be sure of the distinction is to see the other model. In C++ you can declare a reference parameter with `&`, and then assignment inside the function rebinds the caller's variable:

CPP Copy

```
// C++ — genuine pass-by-reference
void replace(User*& p) {
    p = new User("Bob");   // the CALLER's pointer now points at the new o
}
// Java has no syntax for this. There is no way to make a method
// rebind a variable belonging to its caller.
```

That absence is the point: pass-by-reference means the callee can rebind the caller's *variable*. Java offers no mechanism to do that, which is why the answer is unconditional.

Why did Java choose this? It removes an entire class of surprise. When you call `process(order)`, you know your variable `order` will still point at the same object afterwards — the method cannot swap it out from under you. It can change the object's contents, but the binding is yours. This makes local reasoning far easier, and it is the same instinct behind `final` parameters and the general modern preference for immutability.

The practical takeaway is a design one. If you do not want a method mutating what you hand it, do not rely on the calling convention to protect you — it will not. Pass an immutable object, or a defensive copy, or an unmodifiable view (`List.copyOf(list)`). Chapter 3's discussion of immutability and Chapter 10's records are both, in part, answers to this problem.

Common mistakes

- **"Objects are passed by reference."** The single most common wrong answer in Java interviews. The *reference* is passed by value.
- **Expecting a reassignment inside a method to be visible.** It never is. If the method needs to give you a different object, it has to return it.
- **Trying to write `swap`.** It cannot work. Return a new value, or mutate a shared holder.
- **Assuming `final` on a parameter changes anything for the caller.** It only stops the callee rebinding its own slot — which the caller could not see anyway. It does not make the object immutable.
- **Thinking `String` proves objects are copied.** The `String` object is not copied; the reference is. Nothing changes because `String` is immutable, so `+` can only make a new one.
- **Handing out an internal collection and assuming it is safe.** The caller receives a reference to your live list and can clear it. Return an unmodifiable view or a copy.

Interview lens

> **Q: "Is Java pass-by-value or pass-by-reference?"**
> A: Strictly pass-by-value, with no exceptions. For a primitive, the value copied is the number. For an object, the value copied is the reference — so the callee gets its own arrow pointing at the same heap object. That is why mutating the object inside the method is visible to the caller, but reassigning the parameter is not. Pass-by-reference would allow the callee to rebind the caller's variable, and Java has no way to express that.

> **Q: "Then why does modifying an object inside a method affect the caller?"**
> A: Because both variables reference the same object. Copying the reference does not copy the object. Mutation goes through the arrow to the one shared object; the caller sees it because there is only one object. Nothing about that requires pass-by-reference.

A: One method. Take `void replace(User p) { p = new User("Bob"); }`, call it with a `User` named "Asha", and print the caller's name afterwards. It still says "Asha". Under pass-by-reference the caller's variable would have been rebound to the new object. It was not, because the method only rebound its own copy of the arrow.

Q: "Can you write a `swap` method in Java?"

A: Not one that swaps the caller's two variables — there is no way to rebind a caller's variable. The workarounds are to return the pair (an array, a record, a small holder), to swap elements inside a shared mutable container such as `Collections.swap(list, i, j)`, or to pass a mutable holder object and mutate its fields.

Q: "What happens if I pass a `String` and modify it inside the method?"

A: Nothing observable to the caller. `String` is immutable, so `s = s + "!"` creates a new `String` and rebinds the method's own parameter slot. The caller's variable still points at the original. Use a `StringBuilder` if you genuinely need the callee to modify shared text — and then the mutation *is* visible, because `StringBuilder` is mutable.

Q: "Does making a parameter `final` protect the caller's object?"

A: No. `final` on a parameter only prevents the method rebinding that parameter slot — something the caller could never observe anyway. The object stays fully mutable through it. If you want protection, pass an immutable type, a defensive copy, or an unmodifiable view.

Q: "How do you stop a method mutating the collection you pass it?"

A: Do not depend on the calling convention — it gives you nothing. Pass `List.copyOf(list)` for an immutable snapshot, or

returning internal state: hand back a copy or an unmodifiable view rather than the live collection.

Cheat-card

- **Java is pass-by-value. Always.** No exceptions, no "except for objects".
- For objects, the **reference** is what gets copied — not the object.
- **Mutate** through the parameter → caller sees it. **Reassign** the parameter → caller does not.
- The proof: `p = new User( ... )` inside a method leaves the caller's variable untouched.
- **No** `swap` is possible — return a value or mutate a shared container.
- `String` looks like a counter-example but is just immutability: `+` makes a new object.
- Arrays and collections ARE objects — `arr[0] = 9` and `list.clear()` are visible to the caller.
- `final` parameter ≠ immutable object. It only blocks rebinding the local slot.
- Want safety? **Immutable type, defensive copy, or unmodifiable view.**

QUICK CHECK — no marks, just for you.

1. Is Java pass-by-value or pass-by-reference?

- ○ Always pass-by-value — for an object, the value copied is the reference, not the object
- ○ It depends on whether the parameter is declared final
- ○ Pass-by-value for primitives, pass-by-reference for objects
- ○ Pass-by-reference for everything, which is why mutations are visible to the caller

2. static void replace(User p) { p = new User("Bob"); } — after calling replace(u) where u.name is "Asha", what is u.name?

- ◯ It depends on whether User is mutable
- ◯ "Asha" — the method rebound only its own parameter slot; the caller's variable never moved
- ◯ "Bob" — the caller's reference now points at the new object
- ◯ null — the original object became unreachable

Check answer

3. Which of these methods will NOT be visible to the caller?

- ◯ void f(StringBuilder sb) { sb.append("!"); }
- ◯ void f(int[] a) { a[0] = 99; }
- ◯ void f(String s) { s = s + "!"; }
- ◯ void f(List<String> l) { l.add("x"); }

Check answer

Java Interview Kit

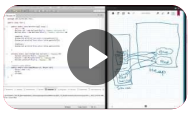
LEARN MORE

ARTICLES & DOCS

Passing information to a method or constructor
docs.oracle.com

Method invocation (JLS §15.12)
docs.oracle.com

VIDEOS



Java Pass By Value Stack Heap Memory Explanation
Diyansh IT Services

Hand-picked references for this lesson.